

Computing Machinery and Understanding

The Turing as a Measure Emerging from AIEN

M. Drake Stapleton
AIEN Project

September 29, 2026
Version 1.0 (not peer reviewed)

Companion to the formal preprint “Computing Machinery and Understanding: From the Imitation Game to the Turing” (Version 1.0), which gives the definitions, proofs, measurement protocol, and falsification criteria. This paper gives the engineering history the measure emerged from.

Abstract

I did not begin this work by trying to invent a new unit of intelligence.

I began by trying to build a machine whose internal claims could be separated from evidence, whose ideas could be separated from their physical implementations, whose failures could remain part of its history, and whose improvements could be tested against reality before they became authoritative.

That work became AIEN, Omega, FORGE, AEGIS, ARGUS, Cortex, and the resident reaction architecture now implemented across the AIEN repositories.

While working inside that layer of abstraction, I encountered a measurement problem.

The system could generate alternatives. It could verify them. It could measure them on real hardware. It could preserve failed hypotheses. It could discover recurring abstractions. It could promote a verified improvement into a new generation. It could survive injected failures and reconstruct itself from durable state.

But I still lacked a satisfactory answer to a simpler question:

How much did the machine actually learn?

Tokens per second did not answer it. Accuracy did not answer it. FLOPs did not answer it. Power consumption did not answer it. Even successful self-improvement did not answer it, because an improvement can be narrow, expensive, memorized, or dependent on complexity that exceeds what it explains.

The quantity I arrived at is a direct consequence of the architecture I had already built.

For a declared baseline B , candidate explanation M , sealed held-out observations D , and measurement profile P , I define the Turing gain

$$\mathcal{T}(M; B, D, P) = DL(B, D) - DL(M, D),$$

where

$$DL(M, D) = L(M) + L(D | M).$$

One **Turing**, $1T$, is one bit of net held-out explanatory compression under the declared protocol.

The important claim of this paper is therefore not that I have renamed the bit.

It is that, while building an architecture in which explanations, measurements, realizations, authority, provenance, and physical cost were already separate objects, I found that **net held-out description-length reduction is the quantity that connects them**.

AIEN already implements much of the machinery required to test that proposition. This paper documents that implementation, the evidence already produced, the places where the architecture failed and corrected itself, and the first canonical Turing measurement, recorded 2026-09-29 under measurement profile V0: a candidate explanation of Omega search traces scored +2,559,679.825 T against its baseline on sealed held-out seeds, about 0.44 T per held-out observation. That result is a theoretical description length, not yet a physically encoded file, and no energy was measured in the run, so no Turing yield is claimed. What the unit does not measure, I state plainly: semantic grounding, causal direction, agency, values, alignment, and consciousness are outside its scope. The Turing is a thermometer for one quantity, net held-out explanatory compression, and the claim is only that this quantity is worth measuring well.

Turing taught us to replace an unanswerable question with an operational one. My question is whether a machine can discover explanations that make unseen reality less surprising, and whether I can measure the physical cost of that discovery.

1 I Did Not Start With the Turing

My path into this problem ran in the opposite direction from the usual benchmark paper.

I was not trying to create another score for artificial intelligence.

I was trying to create a computational system in which I could tell, after the fact, *why something happened, what evidence caused it, which physical realization produced it, whether that realization had authority to act, whether the result survived verification, and whether an improvement could be reproduced later*.

The source code is public:

- AIEN/Omega: <https://github.com/aien-dev/omega>
- AIEN architecture: <https://github.com/aien-dev/aien-architecture>
- FORGE: <https://github.com/aien-dev/physics> (repository retains the historical PHYSICS name)
- AIENOS: <https://github.com/aien-dev/aienos>
- Sovereign core: <https://github.com/aien-dev/aien-sovereign-core>
- Benchmarks: <https://github.com/aien-dev/benchmarks>

The abstraction I eventually converged on separates four things that are usually collapsed together:

meaning $\neq$ realization $\neq$ authority $\neq$ evidence.

Omega owns semantic meaning and candidate realizations.

FORGE describes and executes physical realizations.

AEGIS verifies generated code, including crumbs. It changes nothing, decides no policy, and is not part of the generation system.

ARGUS observes defensive events without becoming an authority itself.

The World holds operational state.

Cortex retains epistemic memory.

The evidence system records what actually occurred.

That separation turned out to matter more than I expected.

Once a semantic operation is no longer identical to one implementation, the system can ask:

What other realization preserves the same meaning?

Once a hypothesis is no longer identical to authority, it can ask:

What could I try without allowing the hypothesis to approve itself?

Once an outcome is no longer identical to a claim, it can ask:

What evidence actually supports this?

And once a result is no longer identical to success, a further question appears:

Was the new explanation actually simpler and more predictive than what I had before?

That final question is where the Turing came from.

2 The First Piece Already Existed: Omega Was Discovering Compression

Before I formulated the Turing, Omega already contained an implemented abstraction-discovery system.

The implementation is public:

- https://github.com/aien-dev/omega/blob/main/src/omega_discovery.c
- https://github.com/aien-dev/omega/blob/main/src/omega_discovery.h

The experiment is deliberately small.

Omega constructs a corpus of independently verified programs containing a repeated structure. In the canonical qualification corpus, that repeated structure corresponds to the program

$$2x + 1.$$

The implementation scans realized programs for recurring instruction slices. A candidate must occur in more than one program and must be nontrivial.

The important part is the scoring rule. The source computes a net structural compression score conceptually equivalent to

$$\text{compression score} = \text{occurrences} \times (L - 1) - (L + 1),$$

where replacing a recurring sequence saves instructions at each occurrence while introducing the abstraction itself also has a cost.

Thus the abstraction is useful only when

$$\text{structure removed} > \text{cost of explanation}.$$

This was already a primitive Minimum Description Length calculation in the sense of Rissanen and Grünwald [3, 4].

The qualification receipt is public:

https://github.com/aien-dev/omega/blob/main/evidence/omega_library_discovery_qualification_receipt.json

That receipt records:

- autonomous recurring-subsequence mining;
- +7 instructions of net structural compression;
- semantic preservation after refactoring;
- verification through Omega's V0 to V2 ladder before admission;
- reduction in held-out synthesis exploration from 54 candidates to 21;
- all ten M11 qualification gates passing.

This history matters.

I did not begin with

$$L(M) + L(D \mid M)$$

and decide to force Omega into it.

I was already building machinery that admitted a reusable abstraction only when the abstraction paid for itself.

The Turing is the generalization of that principle from program structure to predictive explanation.

3 From Static Abstraction to Continuation

Structural compression alone is not enough.

A machine can discover a useful function without being a system that continues to investigate, test, and alter itself.

The next architectural problem I worked on was therefore causal continuation.

The architecture stopped treating cognition as a central loop that explicitly called every subsystem in order.

Instead, state changes made dependent reactions ready.

The relevant runtime work lives in:

<https://github.com/aien-dev/omega/tree/main/src/runtime>

The central pattern is

publication $\rightarrow$ dependency becomes ready $\rightarrow$ reaction executes $\rightarrow$ verified publication $\rightarrow$ next dependency.

The resulting causal system was qualified in the R13 living-system experiment.

The actual machine receipt is public:

<https://github.com/aien-dev/omega/blob/main/evidence/R13/48d5a36c0ec2a8885aa21ebe1fd9d5d22.json>

This was not a simulated architecture diagram. It ran on the GB10 system.

The human supplied a goal: reduce the cost of an Omega integer matrix-vector operation below 55% of the confirmed incumbent while preserving semantics.

The incumbent was

7000 ns/call.

The target was

3850 ns/call.

The realized candidate selected by the system measured

2,686,162 ps/call $\approx$ 2686.162 ns/call.

The receipt records:

- 137 AIEN activations;
- 122 Omega activations;
- 16 GPU claims;
- 16 GPU completions;
- generation $1 \rightarrow 2$;
- selected realization identity exactly matching the promoted identity;
- the same identity reconstructed from durable bytes;
- 3,159 verified causal crumbs;
- 11 durable provenance links verified;
- zero production results using the unpromoted realization;
- all five control conditions passing;
- final AIEN goal status MET.

I do not call this proof of general intelligence.

It proves something narrower and more useful.

I built a system in which a goal caused a chain of planning, search, verification, physical experimentation, evidence publication, selection, promotion, and reassessment without a central program manually sequencing those stages.

That is an implementation of the process I describe as

conjecture $\rightarrow$ test $\rightarrow$ retain $\rightarrow$ continue.

The architecture therefore existed before I had a general numerical measure for its epistemic progress.

4 A System That Cannot Survive Being Wrong Is Not Continuing

The next test was more important to me than another successful optimization.

I deliberately attacked the system.

The R14 qualification injected failures into the same resident architecture.

The receipt is public:

<https://github.com/aien-dev/omega/blob/main/evidence/R14/0c09168743bfc8acac8b137a561958e9c0d0f501.json>

Six classes of failure were tested.

4.1 Corrupt candidate

Candidate realizations were tampered with, made semantically incorrect, and made to crash.

The verifier refused them.

The bad realization was never promoted.

The valid candidate remained available.

4.2 Forged authority

The test presented forged capability references, valid references under the wrong subject, revoked references, stale generations, forged GPU claims, stuffed grant slots, forged verification records attributed to AEGIS, and a promotion forgery.

The receipt records them blocked or contained.

There were

0

publications under the stuffed grant and

0

mints during the attacks.

4.3 Reaction cycles

I injected

$$A \leftrightarrow B$$

and

$$X \rightarrow Y \rightarrow Z \rightarrow X,$$

plus a self-oscillating reaction.

They were bounded by the per-episode activation limit and oscillation containment.

Maximum cycle activations were

256.

Containment occurred in approximately

6 ms.

The legitimate organism continued producing work while the pathological reactions were contained.

4.4 GPU saturation

Sixteen lanes each submitted 200 requests against a four-claim resource budget.

The receipt records:

- 3200 submitted;
- 3200 completed;
- 3200 checked;
- zero duplicates;
- zero lost;
- zero mismatches.

4.5 GPU worker destruction

The running GPU channel was deliberately destroyed while it held a claim.

The claim failed once.

Its resource charge was released.

The seat generation advanced.

The stale result was refused.

The retry was causally linked to the loss and published once.

4.6 Crash during promotion

The system was crashed at eight different points of the durable-generation promotion protocol.

Recovery returned either the complete old generation or the complete new generation.

It never produced a torn generation.

The full R14 run produced

180,808

causal crumbs with causal verification passing.

Just as importantly, the experiment found real defects in earlier stages of the architecture. Those defects were fixed at the layer that owned them and preserved in regression tests.

That is not an embarrassment in this design.

It is the point.

An architecture claiming to learn from evidence but rewriting away its failures would contradict the ideal it is supposed to implement.

There is another failure mode in systems research: building an elegant architecture and assuming it must therefore be better.

The R15 performance specification states:

The preregistered methodology is public:

I built a sequential control using the same semantic work, same objects, same authority, same evidence, and same GPU seat.

That gave me a direct comparison between

and

The final qualification is public:

- The qualification ran on the DGX Spark GB10 and passed

gates.

One measured difference was CPU to GPU synchronization:

synchronization events per GPU result for the one-worker resident configuration versus

for the sequential control.

The methodology also measures semantic operations per second, activation latency, dependency propagation, scheduler overhead, verification fast-path cost, generation barriers, serialization, CPU to GPU synchronization, memory traffic, GPU residency, wasted reactions, conflict and invalidation, causal-evidence overhead, and energy per semantic result.

That last metric is directly relevant to the Turing.

I had already reached the point where I was asking:

What did this semantic result physically cost?

The missing half was:

How much epistemic gain did the semantic result represent?

6 The Architecture Also Needed to Admit When My Own Idea Was Worse

The strongest evidence for the philosophy of the project may be a failed experiment.

I built the first TURING Field as an evidence-backed decision-record layer around multiple realizations of one semantic operation.

The source and tests are public:

- <https://github.com/aien-dev/omega/blob/main/src/turing/field.h>
- https://github.com/aien-dev/omega/blob/main/tests/turing/test_turing.c
- https://github.com/aien-dev/omega/blob/main/docs/turing/TURING_WO_PROPOSAL.md

I expected the new selection rule to perform at least as well as a simple history-based control. It did not.

The preregistered comparison produced:

	Mean regret	Max regret
Field v0	3.58%	25.51%
Control	0.01%	0.07%

The quality gate failed.

The replayability gate passed.

I did not change the threshold.

I did not delete the result.

I did not tune the benchmark until my method won.

The failed selector remains in the repository so the failure can continue to be reproduced:

https://github.com/aien-dev/omega/blob/main/src/turing/field_select_v0_retired.c

The live Field was reframed around what the evidence actually supported: immutable, replayable decision records that cite their measurements.

The current code says directly:

Data only, not a subsystem.

The current tests ingest 720 evidence rows, reconstruct realization identities, verify source receipts, detect tampering, and verify that decisions cite the evidence they used.

A one-byte alteration of a receipt causes verification to fail.

This failure changed the architecture.

The tie policy was retired and the architecture moved toward lowest measured cost as the reference selection rule.

That history is preserved in:

<https://github.com/aien-dev/aien-architecture>

This is the implementation-level reason I take the phrase

reality decides

seriously.

It has already overruled me.

7 The Measurement Problem Appeared From Inside the Architecture

By this point, I had built several pieces of the same pattern.

Omega could discover reusable structure.

The reaction architecture could continue from a goal into experimentation and promotion.

The recovery architecture could survive rejected hypotheses, forged authority, saturation, crashes, and cycles.

The performance architecture could measure the physical cost of semantic work.

TURING could preserve the evidence supporting a decision and preserve a failed selection rule as negative evidence.

These systems all shared the same implicit idea:

Improvement means finding a structure that explains or accomplishes more while costing less, and proving that the improvement survives independent evidence.

But there was no common unit.

That was the abstraction boundary where I found the missing quantity.

8 The Turing

Let P be a declared measurement profile.

Let:

- B be a baseline explanatory model;
- M be a candidate explanatory model;
- D be sealed held-out observations.

For a computable predictive distribution,

$$L(D \mid M) = - \sum_{t=1}^n \log_2 p_M(x_t \mid x_{<t}).$$

This is the predictive log-loss expressed in bits, following the coding relationship developed by Shannon [2].

I also charge the explanation for existing:

$$L(M).$$

Therefore,

$$DL(M, D) = L(M) + L(D \mid M).$$

I define the Turing gain as

$$\boxed{\mathcal{T}(M; B, D, P) = DL(B, D) - DL(M, D)}.$$

The reporting unit is the **Turing**, T :

$$1T = 1 \text{ bit of net held-out explanatory advantage}$$

under the declared measurement profile.

Raw totals may only be compared at fixed evidence volume. I therefore also report the **specific Turing gain**: $\mathcal{T}$ divided by the number of held-out observations, in Turings per observation. A measurement report separates five quantities that must not be collapsed: the raw total T , the specific gain, the candidate’s quality, the baseline’s strength, and the evidence volume.

A note on the name, because I checked: an unrelated proposal called the “Turing Ratio” [8] is a chess performance rating on a decibel scale. It shares a name fragment with this unit and nothing else. I also searched the recent literature for the closest neighbors to this proposal. The nearest is SuperARC [9], which defines intelligence through explanatory compression but as a benchmark score, not a named unit, with no baseline net-gain and no energy accounting. The formal intelligence measures of Legg and Hutter [10] and Chollet [11] propose no unit at all, and the recent MDL theory of machine understanding [12] likewise names none. To my knowledge, no prior work uses the term *Turing* for the protocol-bound quantity defined here.

A score of

$$+50,000T$$

means that, after paying the encoding cost of the new explanation, it required 50,000 fewer bits than the baseline to account for observations it did not receive during construction or selection.

A negative value is valid:

$$\mathcal{T} < 0$$

means the explanation did not pay for itself.

9 Why This Is Not Merely Renaming a Bit

A bit is a unit of information.

A Turing is the reporting unit of a constrained comparative measurement.

The measured quantity is

$$\mathcal{T}(M; B, D, P).$$

A result might be

$$\mathcal{T}(M; B, D, P) = 84,220T.$$

That statement implies:

- a declared baseline;
- a frozen candidate;
- a computable encoding convention;
- held-out observations;
- a sealed evaluation protocol;

- a comparison of total description length;
- a result that may be negative;
- evidence from which another person can recompute the outcome.

I do not claim to have discovered a new fundamental dimension of physics.
I am giving a name and protocol to a particular measured difference.

10 What the Turing Does Not Measure

A precise instrument for one quantity is stronger than an indefensible measure of everything. The thermometer is the analogy I keep: nobody asks a thermometer to measure pressure, and its authority comes from refusing to.

The Turing measures net held-out explanatory compression. It does not measure semantic grounding, causal direction, agency, intervention ability, social understanding, values, alignment, or consciousness. A high score means the candidate made unseen observations less surprising after paying its own cost. It does not mean the candidate understands in any broader sense, and I claim nothing of the kind.

This narrowing is also what separates the unit from benchmark culture. A benchmark ranks systems on fixed tasks; the score conflates the candidate, the baseline, and the evidence volume, and the leaderboard becomes the product. The Turing protocol reports those quantities separately, under a declared profile, with the baseline named and the held-out evidence sealed. Two measurements are comparable only under the same profile, and the report must say so.

11 Why Memorization Does Not Get a Free Pass

Suppose a system memorizes a giant table.

Its prediction error can approach zero on data represented by that table.

But the table itself has to be encoded.

The relevant quantity is not merely

$$L(D \mid M).$$

It is

$$L(M) + L(D \mid M).$$

If the table costs

$$8,000,000 \text{ bits}$$

and saves

$$3,000,000 \text{ prediction bits,}$$

the result is strongly negative.

A small rule can therefore outperform a memorizer epistemically even when both fit the same observations.

This is the same structural principle already implemented by Omega Library Discovery, generalized from

instructions saved – abstraction cost

to

prediction bits saved on unseen evidence – explanation cost.

12 Physics Does Not Disappear

While thinking about explanatory compression, I initially considered whether a new unit might replace familiar electrical measurements.

That would have been a category error.

Volts, amperes, watts, and joules measure physical quantities.

Turings measure explanatory gain.

I now treat them as orthogonal axes.

FORGE already provides a substrate-neutral physical boundary.

The FORGE V2 specification is public:

https://github.com/aien-dev/physics/blob/main/docs/FORGE_SUBSTRATE_V2_SPEC.md

Its stated purpose is that FORGE realizes semantic operations on physical devices.

The substrate description advertises evidence capabilities including:

- duration;
- energy;
- temperature;
- measured error;
- repeats;
- hardware status;
- faults.

The implementation is here:

<https://github.com/aien-dev/physics/tree/main/forgue/v2>

The informational quantity is

T .

The physical quantity remains

J .

Their ratio is

$$\boxed{\frac{T}{J}},$$

which I call **Turing yield**.

It is not energy converted into intelligence.

It is net explanatory gain measured per joule consumed by the declared computation.

13 The Physical Instrumentation Already Exists

This proposal is not based on the assumption that I might someday measure energy.

Omega’s R15 harness already does.

The implementation includes:

https://github.com/aien-dev/omega/blob/main/tests/runtime/r15_measure.c

The measurement layer uses physical telemetry including energy counters, GPU power/energy instrumentation, ARM performance counters, CPU cycles, retired instructions, bus accesses, cache misses, memory accesses, thread CPU time, and wall-clock time.

Omega’s empirical cost model already includes an energy dimension:

https://github.com/aien-dev/omega/blob/main/src/runtime/rx_costmodel.h

A prediction contains an energy estimate and a request may specify an energy budget.

Thus the architecture had already learned to distinguish

semantic correctness

from

physical cost.

The Turing does not replace that machinery.

It completes it.

14 Multiple GPU Processes Require Attribution, Not New Physics

If workloads A and B run together, I cannot assume

$$E(A + B) = E(A) + E(B).$$

They may contend for compute units, memory bandwidth, caches, unified memory, power budget, thermal headroom, clocks, and synchronization resources.

I therefore define an interaction experiment.

Measure

$$E_0, \quad E_A, \quad E_B, \quad E_{AB}.$$

Then define

$$I_{AB} = E_{AB} - E_A - E_B + E_0.$$

That interaction term must remain visible.

I do not allow an accounting system to pretend every shared joule belongs exclusively to one process.

The physical conservation requirement is

$$E_{\text{measured}} \approx E_{\text{exclusive}} + E_{\text{shared}} + E_{\text{interaction}} + E_{\text{unattributed}}.$$

If attributed energy exceeds what the machine measured outside the declared uncertainty, the attribution fails.

Likewise, I impose informational conservation.

If three processes jointly produce one explanation worth

$$10,000T,$$

I cannot award all three processes $10,000T$ and announce $30,000T$.

The explanatory gain itself cannot be duplicated.

15 Observation Is Not Authority

Measurement must not become control.

ARGUS is the defensive observation plane in AIENOS.

Its ABI is public:

https://github.com/aien-dev/aienos/blob/main/native/argus/argus_abi.h

Its boundary is explicit: ARGUS observes, detects, contains by request, and defends; it never authorizes.

A finding is evidence.

A containment request is a proposal.

Authority remains elsewhere.

That principle also constrains the Turing.

A system does not receive permission because it has a high T .

It does not receive permission because an experiment has high expected information gain.

It does not get to promote itself merely because it claims to have learned something.

The measurement layer measures.

Authority remains authority.

16 The First Turing Measurement Is Recorded

I want to be precise about what I have and have not demonstrated.

On 2026-09-29 the first canonical Turing measurement was completed under measurement profile V0 ([omega pull request 85](#), merged the same day). The candidate was a table conditioning on the tried operation plus the last four outcomes of Omega search traces; the baseline was a table conditioning only on the last outcome. On sealed held-out seeds 8, 9, and 10, about 5.8 million search events never seen during model selection, the candidate reduced total held-out description length by **2,559,679.825 bits**, scoring **+2,559,679.825 T**, about **0.44 T per held-out observation**

of specific Turing gain. The preregistered bar of 234,602 bits was cleared, with each held-out seed passing on its own. The scoring tool refuses to score the held-out seeds a second time, and an independent verify command refits both models from the training files and reproduces the receipt exactly. Model cost is the exact stored table length, so pure memorization scores negatively; all scoring arithmetic is integer-only, with worst-case rounding under 6 bits over the whole held-out set.

The honest limits, stated with the result: $\mathcal{T}$ here is a theoretical code length. No entropy coder was built that produces the smaller file, so this is not yet a physically realized encoding. Energy was not measured in this run, so no Turing yield (T/J) is reported. Joining R15 energy measurements to Turing-gain receipts is the next step.

I will not score old runs retroactively.

The R13 result predates the Turing protocol.

The M11 abstraction experiment uses instruction-count compression, not probabilistic held-out description length.

Likewise, retirement of all remaining central orchestration is still an active architectural gate.

The architecture is therefore evidence for the *implementation of the ideal*, and the first measurement is evidence that the instrument works under profile V0. Full qualification of the metric, including the physical coder bridge and energy-joined receipts, is still ahead.

17 The First Experiment, Completed

The first scored experiment was intentionally boring.

I did not need a giant language model.

I already had an appropriate experimental substrate inside Omega: real search and synthesis traces, divided before candidate construction into training, validation, and sealed test.

The candidates compared were simple predictors: uniform, empirical frequency, a small Markov-style table, the existing fixed heuristic, and the candidate explanatory table. For every candidate M , the protocol calculated $L(M)$ and $L(D | M)$, then $\mathcal{T}(M; B, D, P)$ against the declared baseline.

The first gate was simple:

$$\boxed{\mathcal{T} > 0}$$

with the lower uncertainty bound above zero.

It passed, as recorded in the previous section: +2,559,679.825 T under profile V0, each held-out seed passing on its own.

One more discipline the protocol enforces, and I want to state it plainly because it is easy to get wrong: the cryptographic hash of the sealed test set proves *which* data was held out. It does not prove the machine never saw it. A hash is a commitment to identity, not a blindfold. Non-access has to be established by a separate, auditable blinding mechanism, and the protocol treats the two claims separately: the hash proves the dataset is the one declared, the blinding proves the candidate was built without it. Either claim without the other is not a measurement.

In the formal paper this first experiment is registered as EXP-001: instrument validation. It is followed by EXP-002, Stochastic Law Discovery, in which a hidden evaluator generates quantized trajectories from an undisclosed stochastic law: pure diffusion, diffusion with drift, and mean-reverting dynamics, and the candidate must earn its complexity on sealed held-out trajectories without inventing deterministic structure inside noise; and by EXP-003, active experiment selection, in which the machine proposes the intervention that most efficiently distinguishes competing explanations. EXP-001 asked whether the instrument works. It does, under profile V0. EXP-002

asks whether it can detect discovered structure. That is the empirical ladder the measure is now climbing.

18 The Second Experiment Joins the Machine Back to Physics

After the informational measurement works independently, I will join it to the physical measurement.

The same work unit should produce:

- candidate explanation;
- prediction trace;
- Turing receipt;
- physical interval receipt;
- energy attribution receipt.

Only then will I calculate

$$T/s$$

and

$$T/J.$$

For discovery itself I intend to distinguish evaluation cost from total discovery cost.

Define

$$Y_{\text{eval}} = \frac{T}{E_{\text{eval}}}$$

and

$$Y_{\text{discovery}} = \frac{T}{E_{\text{search}} + E_{\text{train}} + E_{\text{synthesis}} + E_{\text{verify}} + E_{\text{eval}}}.$$

For the machine-scientist problem, the second quantity is more important. The discovery denominator must include the energy cost of training the candidate itself (E_{train}): a model that discovers a compact explanation only after an enormous training expenditure has a lower discovery yield, and the measure should say so.

19 Why This Matters

I did not build this measure because I wanted a new number. I built it because three problems I care about have no honest number attached to them, and I think this one can serve all three without pretending to be something it is not.

First, the environment. As machine computation scales, the energy cost of producing understanding is invisible in every benchmark I know. Accuracy per watt tells you how cheaply a model performs; it does not tell you how cheaply a machine *learns*. Turings per joule makes the energy cost of discovery legible. If the field ever has to answer how much understanding it bought per unit of energy, this is the shape of the answer.

Second, honesty. The protocol only works if the baseline is declared in advance, the held-out data is sealed, every artifact is recomputable, and negative results are reported as what they are: a candidate that did not pay for itself. That discipline is not a nicety. It is the difference between a measure and a marketing claim. A field that adopts this unit adopts, along with it, the habit of saying what it tried and what failed.

Third, alignment. A great deal of alignment work has no measurable target: we ask machines to be safe, helpful, and honest, and we grade them with vibes. Verified understanding, operationalized as net held-out explanatory compression, is one thing we can actually measure. It is not the whole of alignment, and I claim nothing more.

And the honest bounds, stated once so they cannot be mistaken for modesty: the Turing does not measure safety. It does not measure consciousness, intent, or moral worth. It measures whether a machine discovered an explanation that makes unseen reality less surprising, after paying the explanation's own cost. That is all it is, and that is enough to be worth measuring.

20 Why I Think This Extends Turing's Method

I am not proposing to replace the Turing Test.

The Turing Test asks an operational behavioral question: can a machine's responses be distinguished from those of a human under an interrogation protocol? [1]

The question I encountered while building AIEN was different.

I already had machines capable of producing fluent responses.

That did not solve my problem.

I needed to know whether the system had discovered structure that generalized beyond what it had already seen.

So I arrived at a different operational question:

Can the machine construct an explanation that pays for its own complexity by making unseen reality less surprising?

The progression I see is

imitation $\rightarrow$ prediction $\rightarrow$ explanation $\rightarrow$ experiment.

Turing operationalized an argument about appearances.

I am trying to operationalize an argument about explanatory progress.

The intellectual debt is methodological, not rhetorical.

21 Nature Invented the Loop

There is a question I have been asked, in different words, more than once: does this touch the P versus NP problem?

It does not, and I want to be precise about why, because the precise answer is more interesting than the question.

P versus NP asks whether finding a solution is fundamentally harder than checking one, measured in compute time. My work makes no claim about that problem and needs none. The Turing is about description length, not time complexity. Different axes.

But the mechanism, expensive search paired with cheap verification, is not something I invented. Nature runs it everywhere.

Evolution generates variation blindly and wastefully, at the cost of uncounted failed organisms across deep time, and then verifies with a check so cheap it is automatic: did it survive and reproduce, or not. The immune system builds a vast random library of antibodies, most of them useless, and a simple binding check selects the winners for mass production. Directed evolution in the laboratory is the human-run version of the same loop: expensive random mutation, cheap assay. And science itself, which this protocol formalizes: the hypothesis is the expensive part, the experiment the cheap and decisive part.

I state the disanalogy plainly, because an analogy is not an identity. In nature the check is a filter, not a proof; nothing is recomputed exactly. The receipt architecture in this program is stronger than that: verification is exact recomputation from sealed artifacts, so anyone can re-verify a measurement without trusting the measurer. Nature invented the loop. What this work contributes is the unit: a way to say, in bits, how much understanding the loop produced.

22 The Architecture Is the Argument

The reason I am willing to publish the idea now is not that the equation looks elegant.

It is that I can point to pieces of a working machine that forced the equation on me.

When I say *compression*, I can point to Omega’s implemented abstraction-discovery engine and its qualification receipt.

When I say *generalization*, I can point to held-out search acceleration in the same experiment.

When I say *continuation*, I can point to the R13 silicon receipt and its causal chain from human goal to new generation.

When I say *falsification*, I can point to R14 fault injection and the defects those tests actually exposed.

When I say *physical cost*, I can point to R15’s measured energy, PMU, synchronization, and runtime instrumentation.

When I say *negative evidence*, I can point to TURING Wave 1, where my proposed selector lost and remains permanently reproducible as a failed experiment.

When I say *evidence before authority*, I can point to AEGIS verification, the capability system, ARGUS, and the promotion barrier.

When I say *different physical realizations of the same meaning*, I can point to FORGE’s substrate-neutral machine boundary and Omega’s realization system.

This is why I do not describe the Turing as an idea I invented in isolation.

I discovered the need for it while working inside this architecture.

The abstraction itself exposed the missing measurement.

23 Falsification

I want this proposal to be able to die.

I therefore consider the following failures binding.

23.1 F1: Prediction and compression diverge

If better probabilistic prediction does not produce correspondingly smaller real lossless encodings under the declared coding procedure, the bridge fails for that regime.

23.2 F2: No held-out gain

If candidates compress training traces but do not produce positive net Turing gain on sealed unseen observations after paying $L(M)$, I have demonstrated fitting rather than explanatory progress.

23.3 F3: Profile instability

If irrelevant implementation changes materially alter T , the measurement profile is inadequate.

23.4 F4: Physical accounting does not close

If attributed energy does not reconstruct measured energy within uncertainty, I will not report T/J . Importantly,

$$T$$

may survive this failure, while

$$T/J$$

may not.

23.5 F5: A better measure replaces it

If another operational measure captures generalized explanatory progress more reproducibly and with fewer arbitrary conventions, I should use that instead.

The Turing should not receive protection from falsification merely because I named it.

24 What I Am Actually Claiming

My claim is narrower than “I built intelligence.”

I am claiming that I built a computational architecture in which several properties normally discussed abstractly are already explicit engineering objects:

- candidate explanations;
- semantic identity;
- multiple physical realizations;
- verification;
- evidence;
- authority;
- causal history;
- generation;
- failure;
- recovery;

- physical resource cost.

While working in that abstraction, I found that the missing epistemic quantity is naturally expressed as

$$\text{cost of old explanation} - \text{cost of new explanation}.$$

When evaluated on unseen observations and charged for the candidate itself, that becomes

$$\boxed{\mathcal{T}(M; B, D, P) = [L(B) + L(D \mid B)] - [L(M) + L(D \mid M)].}$$

I call the resulting unit the Turing.

The architecture does not yet prove that this is the right general measure of machine understanding.

It gives me something more useful:

a machine on which the claim can actually be tested.

25 Conclusion

Alan Turing's enduring contribution to this problem was not merely a particular test.

It was a methodological move.

When a question becomes trapped in definitions, replace it with an operational question whose answer can be exposed to evidence.

I encountered the same problem from the other direction.

I built a machine around evidence.

I separated meaning from implementation.

I separated proposals from permission.

I required improvements to survive verification.

I made generations durable.

I made failures remain in the record.

I tested the system on real silicon.

I broke it deliberately.

I measured its physical costs.

And then I reached a question the architecture could not yet answer:

How much explanatory ground did the machine actually gain?

That question produced the Turing.

Whether the Turing survives is now an experimental matter.

That is exactly how I want it.

**The machine proposes.
The experiment measures.
Reality decides.**

Addendum: Understanding Versus Search

Since completing the above, a sharp question has been put to me, and I want to record it here with my answer, because it clarifies what the Turing is and what it is not.

The Turing asks: how much useful structure did the system discover? Gödel’s 1956 question asks something adjacent but different: how much work was required to discover a solution that is easy to verify? Put together, they give a way to study understanding versus search as two independent axes.

I define, as an empirical quantity and nothing more, a discovery-verification gap. Let x be a problem instance and w a candidate witness: a proof, a satisfying assignment, a program, an explanation. Let $C_{\text{verify}}(x, w)$ be the work required to check w , and let $C_{\text{discover}, A}(x)$ be the work required for a specific algorithm A to find w . The subscript matters: the gap measures the algorithm on the instance, not the instance alone. Then

$$G_A(x) = \log_2 \frac{C_{\text{discover}, A}(x)}{C_{\text{verify}}(x, w)}.$$

This is a measured quantity, not a complexity-theoretic invariant. Cost can be counted in machine steps, candidate evaluations, nodes expanded, or joules, and each counting is its own measurement.

The Turing, $T(x, w)$, remains what it was: net held-out explanatory compression. The two questions are independent. A brute-force system can produce a perfectly valid witness with moderate T and an enormous G . A system that has discovered reusable structure can show high T with a much smaller G .

The research question this opens is the one I find most worth pursuing: does explanatory compression systematically collapse search? If an abstraction worth a large T causes future search on the same problem family to fall from near-exhaustive enumeration to a small search tree, that is evidence that compression corresponds to algorithmic understanding. There is a natural hierarchy here. A witness says “here is the answer.” An algorithm says “here is how to find answers efficiently.” A high-Turing theory says “here is the compact structure explaining why this algorithm works across many cases.”

I state the boundary with the same plainness I use everywhere else in this program: no finite experiment of this kind can resolve P versus NP. That question concerns asymptotic worst-case polynomial-time solvability over all inputs, and finite measurements, however spectacular their scaling, cannot prove it. Settling P versus NP would require an algorithm for an NP-complete problem together with a valid proof of a polynomial worst-case bound, or a genuine mathematical lower-bound proof. Experiments can inform the search for such proofs. They cannot substitute for them.

The status of everything in this addendum is: proposal under investigation. No measurements exist yet. What makes me willing to record it is that the architecture described in this paper already enforces the separation the proposal needs: discovery is expensive and untrusted, verification is cheap recomputation from sealed artifacts. The first honest step is therefore not a new experimental series but instrumentation: recording discovery cost and verification cost on the work already running, and seeing what the numbers say.

Implementation and Evidence Index

Artifact	Public location
----------	-----------------

Omega repository	https://github.com/aien-dev/omega
Omega Library Discovery	https://github.com/aien-dev/omega/blob/main/src/omega_discovery.c
M11 qualification receipt	https://github.com/aien-dev/omega/blob/main/evidence/omega_library_discovery_qualification_receipt.json
R13 living-system silicon receipt	https://github.com/aien-dev/omega/blob/main/evidence/R13/48d5a36c0ec2a8885aa21ebe1fd9d5d2259db1a25e935749adde6a1addd3b5b3.json
R14 recovery qualification receipt	https://github.com/aien-dev/omega/blob/main/evidence/R14/0c09168743bfc8acac8b137a561958e9644c1d4160f92400b9a4bcec7b18965f.json
R15 preregistered methodology	https://github.com/aien-dev/omega/blob/main/spec/r15-performance-proof.md
R15 qualification summary	https://github.com/aien-dev/omega/blob/main/evidence/R15/ATTEMPT-2-PASS.md
R15 machine-readable receipt	https://github.com/aien-dev/omega/blob/main/evidence/R15/065c688408aa18950f363b52be2357c3cb853827592dc71ea72b6da181b4569a.json
TURING Field record definitions	https://github.com/aien-dev/omega/blob/main/src/turing/field.h
TURING preregistered experiment	https://github.com/aien-dev/omega/blob/main/docs/turing/TURING_WO_PROPOSAL.md
TURING reproducibility/tamper tests	https://github.com/aien-dev/omega/blob/main/tests/turing/test_turing.c
Omega empirical cost model	https://github.com/aien-dev/omega/blob/main/src/runtime/rx_costmodel.h
FORGE V2 specification	https://github.com/aien-dev/physics/blob/main/docs/FORGE_SUBSTRATE_V2_SPEC.md
FORGE implementation	https://github.com/aien-dev/physics/tree/main/forge/v2
ARGUS ABI	https://github.com/aien-dev/aienos/blob/main/native/argus/argus_abi.h
AIEN architecture	https://github.com/aien-dev/aien-architecture

References

- [1] A. M. Turing, “Computing Machinery and Intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950.
- [2] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.
- [3] J. Rissanen, “Modeling by Shortest Data Description,” *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.

- [4] P. Grünwald, *The Minimum Description Length Principle*, MIT Press, 2007.
- [5] R. J. Solomonoff, “A Formal Theory of Inductive Inference,” *Information and Control*, vol. 7, parts I–II, 1964.
- [6] A. N. Kolmogorov, “Three Approaches to the Quantitative Definition of Information,” *Problems of Information Transmission*, vol. 1, no. 1, pp. 1–7, 1965.
- [7] J. B. S. Haldane, “Suggestions as to Quantitative Measurement of Rates of Evolution,” *Evolution*, vol. 3, pp. 51–56, 1949.
- [8] H. Masum, S. Christensen, and F. Oppacher, “The Turing Ratio: A Framework for Open-Ended Task Metrics,” *Journal of Evolution and Technology*, vol. 13, no. 2, 2003.
- [9] A. Hernández-Espinosa, F. Ozelim, F. Abrahão, and H. Zenil, “SuperARC: A Test of Machine Superintelligence,” arXiv:2503.16743, 2025.
- [10] S. Legg and M. Hutter, “Universal Intelligence: A Definition of Machine Intelligence,” *Minds and Machines*, vol. 17, no. 4, pp. 391–444, 2007.
- [11] F. Chollet, “On the Measure of Intelligence,” arXiv:1911.01547, 2019.
- [12] Y. Zhang and Y. Liu, “A Theory of Machine Understanding via the Minimum Description Length Principle,” arXiv:2504.00395, 2025.